Mathematical Exposition of CoDA-NO

Co-Domain Attention Neural Operator — Section 3: Method

Pages 4–7 of the CoDA-NO paper

This document provides a complete, self-contained explanation of every mathematical symbol, operator, and derivation appearing in Section 3 (Method) of the CoDA-NO paper, following the order in which they appear.

Contents

Foundational Setup and Notation

The Input Domain

Symbol: $\mathcal{D}$

$$\mathcal{D} \subset \mathbb{R}^n$$

$\mathcal{D}$ is the **input domain** — a compact subset of n-dimensional Euclidean space on which all physical fields (functions) are defined. For a 1-D PDE solved on the unit interval, $\mathcal{D} = [0, 1]$; for a 2-D problem, $\mathcal{D} \subset \mathbb{R}^2$.

The Input Function and Its Codomain

Symbol: $a : \mathcal{D} \to \mathbb{R}^{d_{\text{in}}}$

$$a : \mathcal{D} \to \mathbb{R}^{d_{\text{in}}}, \qquad a = \left[a^1, \ldots, a^{d_{\text{in}}}\right], \quad a^i : \mathcal{D} \to \mathbb{R}$$

a	The **input function**. It maps every spatial point $x \in \mathcal{D}$ to a vector of d_{in} real numbers representing the physical state of the PDE system at that point.
d_{in}	The number of **physical variables** (channels) in the input. For example, for a fluid problem, $d_{\text{in}} = 3$ could represent velocity u, pressure p, and density ρ.
$\mathbb{R}^{d_{\text{in}}}$	The **codomain** of a — the d_{in}-dimensional real vector space. The paper uses the term *codomain* to mean the range-space of an input function, contrasting it with the domain $\mathcal{D}$.
a^i	The i-th **scalar component** of a, corresponding to the i-th physical variable. Each $a^i : \mathcal{D} \to \mathbb{R}$ is a real-valued field.
$[\cdot]$	**Concatenation** along the codomain axis.

The Output Function

Symbol: $u : \mathcal{D} \to \mathbb{R}^{d_{\text{out}}}$

$$u : \mathcal{D} \to \mathbb{R}^{d_{\text{out}}}$$

u is the **solution (output) function** of the PDE, mapping $\mathcal{D}$ to $\mathbb{R}^{d_{\text{out}}}$. The goal of a neural operator is to learn the *solution operator* $a \mapsto u$.

Neural Operator Primitives

Equation (1) — The Pointwise Operator $\mathcal{H}$

Equation (1): Pointwise Operator

$$\mathcal{H} : \{f : \mathcal{D} \to \mathbb{R}^{d_f}\} \longrightarrow \{g : \mathcal{D} \to \mathbb{R}^{d_g}\}, \qquad \mathcal{H}[f](x) = h_\theta\big(f(x)\big). \tag{1}$$

Symbols in Equation (1)

$f : \mathcal{D} \to \mathbb{R}^{d_f}$	An arbitrary input function with d_f channels.
$g : \mathcal{D} \to \mathbb{R}^{d_g}$	The output function with d_g channels.
d_f, d_g	Input and output channel counts of the pointwise map.
$h_\theta : \mathbb{R}^{d_f} \to \mathbb{R}^{d_g}$	A finite-dimensional parameterised function (typically a neural network such as an MLP) that maps one *codomain vector* to another. It has learnable **parameters** θ.
θ	The learnable parameter vector of h_θ.
x	A point in the domain $\mathcal{D}$.
$\mathcal{H}[f](x)$	The value of the output function at x, obtained by applying h_θ to the value of f at x.

Intuition. A pointwise operator applies the same finite-dimensional map h_θ independently at every location x. It does *not* mix information across different spatial locations; it only mixes information across the channels at a single point.

Equation (2) — The Integral Operator $\mathcal{T}$

Equation (2): Integral Operator

$$\mathcal{T} : \{f : \mathcal{D} \to \mathbb{R}^{d_f}\} \longrightarrow \{g : \mathcal{D} \to \mathbb{R}^{d_g}\}, \qquad \mathcal{T}[f](x) = \int_{\mathcal{D}} k_\phi(x, y)\, f(y)\, \mathrm{d}y. \tag{2}$$

Symbols in Equation (2)

$k_\phi(x, y)$	The **kernel function** (also called the Green's function kernel), mapping a pair of points $(x, y) \in \mathcal{D} \times \mathcal{D}$ to a $d_g \times d_f$ matrix. It has learnable parameters ϕ.
ϕ	The learnable parameters of the kernel k_ϕ.
y	The **integration variable**, ranging over all of $\mathcal{D}$.
$\mathrm{d}y$	The Lebesgue (volume) measure on $\mathcal{D}$.
$k_\phi(x, y)\, f(y)$	A matrix-vector product: the $d_g \times d_f$ kernel matrix multiplied by the d_f-vector $f(y)$, yielding a d_g-vector.
$\int_{\mathcal{D}} \cdots \mathrm{d}y$	Integration over the entire domain. This makes $\mathcal{T}$ a **non-local** operator: the output at x depends on the values of f at *all* points in $\mathcal{D}$.

Intuition. The integral operator generalises convolution to non-uniform domains. The FNO (Fourier Neural Operator) implements $\mathcal{T}$ efficiently in Fourier space for uniform grids: $k_\phi(x, y) = \mathcal{F}^{-1}[R_\phi](x - y)$, where R_ϕ is a learnable Fourier mode tensor.

Multi-PDE Generalisation — Problem Statement

Two PDE Systems

Cross-system notation

$$a : \mathcal{D} \to \mathbb{R}^{d_{\text{in}}}, \quad \bar{a} : \mathcal{D} \to \mathbb{R}^{\bar{d}_{\text{in}}}, \qquad d_{\text{in}} \neq \bar{d}_{\text{in}}$$

$$u : \mathcal{D} \to \mathbb{R}^{d_{\text{out}}}, \quad \bar{u} : \mathcal{D} \to \mathbb{R}^{\bar{d}_{\text{out}}}$$

a, $\bar{a}$	Input functions of two *different* PDE systems. They may have different numbers of physical variables ($d_{\text{in}} \neq \bar{d}_{\text{in}}$).
u, $\bar{u}$	Corresponding output/solution functions with (possibly different) output dimensions.
$\mathcal{G}$	A **neural operator architecture** designed to handle both systems despite the differing codomains.
$\{a^i\}_{i=1}^{d_{\text{in}}} \cap \{\bar{a}^i\}_{i=1}^{\bar{d}_{\text{in}}} \neq \emptyset$	The two systems share at least one **overlapping physical variable**, enabling knowledge transfer.

Neural Operator on Sets — Permutation Equivariance

Equation (3) — Permutation-Equivariant Integral Operator $\mathcal{I}_{per}$

Equation (3): $\mathcal{I}_{per}$

$$\mathcal{I}_{per}[w] = \Big[\mathcal{I}[w_e^1],\ \ldots,\ \mathcal{I}[w_e^{d_{\text{in}}}]\Big]. \tag{3}$$

Symbols in Equation (3)

w	The full latent function, decomposed as $w = [w_e^1, \ldots, w_e^{d_{\text{in}}}]$. Each block w_e^i corresponds to one physical variable.
$w_e^i : \mathcal{D} \to \mathbb{R}^D$	The **group of codomains** (sub-function) associated with the i-th physical variable. D is the latent (lifted) channel dimension per variable.
D	The **lifted** channel width per physical variable in latent space. Must satisfy $D > d_{\text{en}} + 1$ (see Eq. (8)).
$\mathcal{I}$	A *shared* regular integral operator (Eq. (2)), applied independently to each w_e^i.
$\mathcal{I}_{per}$	The **permutation-equivariant integral operator**. Because the same $\mathcal{I}$ is applied to each component independently, permuting the order of physical variables in w yields the same permutation in the output — i.e., the operator is equivariant to re-ordering of physical variables.
$\mathcal{H}_{per}$	Analogous permutation-equivariant *pointwise* operator, using a shared $\mathcal{H}$.
FNO_{per}, GNO_{per}	Permutation-equivariant versions of the Fourier Neural Operator and Graph Neural Operator, respectively, built using the shared-weight mechanism of Eq. (3).

Why permutation equivariance matters. The set of physical variables $\{a^1, a^2, \ldots\}$ has no canonical ordering. A model that is permutation-equivariant will produce equivalent results regardless of the order in which variables are fed in, making it naturally extensible to PDE systems with more or fewer variables.

The CoDA-NO Layer — Codomain Attention

Token Functions

Token Functions

$$w^j : \mathcal{D} \to \mathbb{R}^{d'}, \quad w^j \in \mathcal{W}, \quad j \in \{1, \ldots, T\}, \qquad w = [w^1, \ldots, w^T], \quad d' = \frac{d}{T}$$

w^j	The j-th **token function** — a function from the domain $\mathcal{D}$ to a d'-dimensional real vector. Each token represents one physical variable in the attention mechanism.
$\mathcal{W}$	The **token function space** to which each w^j belongs.
T	The total number of tokens (physical variables) in the current layer.
j	Index running over tokens: $j \in \{1, \dots, T\}$.
d	Total codomain dimension of w.
$d' = d/T$	Per-token channel dimension. By default $d' = 1$ (each physical variable is a scalar field).
$w = [w^1, \dots, w^T]$	**Codomain-wise concatenation** of all tokens into the full latent function $w : \mathcal{D} \to \mathbb{R}^d$.

Equation (4) — Key, Query, and Value Operators

Equation (4): Key/Query/Value Operators

$$\mathcal{K} : \mathcal{W} \to \{k^j : \mathcal{D} \to \mathbb{R}^{d_k}\}, \quad \mathcal{Q} : \mathcal{W} \to \{q^j : \mathcal{D} \to \mathbb{R}^{d_q}\}, \quad \mathcal{V} : \mathcal{W} \to \{v^j : \mathcal{D} \to \mathbb{R}^{d_v}\}. \tag{4}$$

with $d_k = d_q$, and

$$k^j = \mathcal{K}[w^j], \quad q^j = \mathcal{Q}[w^j], \quad v^j = \mathcal{V}[w^j].$$

Symbols in Equation (4): Key/Query/Value

$\mathcal{K}$	The **key operator**, a learned functional mapping each token function w^j to a *key function* $k^j : \mathcal{D} \to \mathbb{R}^{d_k}$. Implemented as an FNO.
$\mathcal{Q}$	The **query operator**, mapping w^j to a *query function* $q^j : \mathcal{D} \to \mathbb{R}^{d_q}$. Implemented as an FNO.
$\mathcal{V}$	The **value operator**, mapping w^j to a *value function* $v^j : \mathcal{D} \to \mathbb{R}^{d_v}$. Implemented as an FNO.
$k^j : \mathcal{D} \to \mathbb{R}^{d_k}$	The key function for token j; a function over $\mathcal{D}$, not a finite vector.
$q^j : \mathcal{D} \to \mathbb{R}^{d_q}$	The query function for token j.
$v^j : \mathcal{D} \to \mathbb{R}^{d_v}$	The value function for token j.
$d_k = d_q$	Key and query dimensions must match for the inner product in Eq. (5) to be defined.
d_v	Value function output dimension.

Extension of standard attention. In classical transformer attention, keys, queries, and values are finite-dimensional vectors. Here they are *functions* over $\mathcal{D}$, making this an infinite-dimensional generalisation that operates in function space.

Equation (5) — Codomain Attention Output

Equation (5): Single-Head Attention Output

$$o^j = \mathrm{Softmax}\left(\begin{bmatrix} \frac{\langle q^j,\, k^1\rangle}{\tau} \\ \vdots \\ \frac{\langle q^j,\, k^T\rangle}{\tau} \end{bmatrix}\right) \begin{bmatrix} v^1 \\ \vdots \\ v^T \end{bmatrix}^\top . \tag{5}$$

Symbols in Equation (5)

$o^j : \mathcal{D} \to \mathbb{R}^{d_v}$	The **output token function** for the j-th physical variable after one attention step.
$\langle q^j, k^m\rangle$	The L^2 **inner product** between the query of token j and the key of token m: $\langle q^j, k^m\rangle = \int_{\mathcal{D}} \langle q^j(x),\, k^m(x)\rangle \,\mathrm{d}x \in \mathbb{R}.$ This integrates the pointwise inner product over the whole domain, producing a single scalar per pair (j, m).
$\int_{\mathcal{D}} \langle q^j(x), k^m(x)\rangle \,\mathrm{d}x$	Explicit form of the $L^2(\mathcal{D}, \mathbb{R}^{d_k})$ dot product. In practice, discretised as a quadrature sum over grid points.
τ	The **temperature** hyperparameter (a positive scalar). Dividing by τ controls the sharpness of the softmax distribution: large $\tau \Rightarrow$ uniform attention; small $\tau \Rightarrow$ peaked attention.
$\mathrm{Softmax}(\cdot)$	The standard softmax function applied to the T-dimensional vector of scaled inner products: $\mathrm{Softmax}(\mathbf{s})_m = \frac{e^{s_m}}{\sum_{m'=1}^{T} e^{s_{m'}}}, \quad \mathbf{s} \in \mathbb{R}^T.$ The output is a probability vector $\boldsymbol{\alpha}^j \in \Delta^{T-1}$ (the $(T-1)$-simplex) of attention coefficients.
$\boldsymbol{\alpha}^j = (\alpha_1^j, \ldots, \alpha_T^j)$	(Implicit.) Attention weights for query j; each $\alpha_m^j \geq 0$ and $\sum_m \alpha_m^j = 1$.
$[v^1, \ldots, v^T]^\top$	Stack of value functions; the weighted sum $o^j = \sum_{m=1}^{T} \alpha_m^j\, v^m$ is a convex combination of value *functions* (not vectors), so o^j is itself a function $\mathcal{D} \to \mathbb{R}^{d_v}$.

Derivation of Eq. (5) step by step.

Step 1. Compute T scalar attention logits for query j:

$$s_m^j = \frac{\langle q^j, k^m\rangle}{\tau}, \quad m = 1, \ldots, T.$$

Step 2. Normalise via softmax to get attention weights:

$$\alpha_m^j = \frac{\exp(s_m^j)}{\sum_{m'=1}^{T} \exp(s_{m'}^j)}.$$

Step 3. Compute the attended output as a weighted sum of value functions:

$$o^j(x) = \sum_{m=1}^{T} \alpha_m^j \, v^m(x), \quad \forall x \in \mathcal{D}.$$

Each α_m^j is a *scalar* measuring how much token j attends to token m (i.e., how strongly the j-th physical variable "looks at" the m-th). The value functions v^m carry the actual information aggregated.

Equation (6) — Multi-Head Mixing Operator $\mathcal{M}$

Equation (6): Multi-Head Mixing

$$\mathcal{M} : \left\{ c^j : \mathcal{D} \to \mathbb{R}^{H \cdot d_v} \right\} \longrightarrow \left\{ o^j : \mathcal{D} \to \mathbb{R}^{d_v} \right\}. \tag{6}$$

Symbols in Equation (6)

H	The number of **attention heads**.
$h \in \{1, \ldots, H\}$	Head index.
$\mathcal{K}^h, \mathcal{Q}^h, \mathcal{V}^h$	Separate key, query, value operators for head h. Each head learns different attention patterns.
$o^{j,h}$	Single-head output for token j from head h, obtained by applying Eq. (5) with operators $\mathcal{K}^h, \mathcal{Q}^h, \mathcal{V}^h$.
$c^j := [o^{j,1}, \ldots, o^{j,H}]$	**Concatenation** of outputs from all H heads along the channel dimension for token j; $c^j : \mathcal{D} \to \mathbb{R}^{H \cdot d_v}$.
$\mathcal{M}$	A learned pointwise operator (implemented as an MLP or linear map) that projects the concatenated $H \cdot d_v$ channels back to d_v channels, mixing the information from all heads.
o^j	The final per-token output of multi-head attention, $o^j : \mathcal{D} \to \mathbb{R}^{d_v}$.

Full Attention Output and Completion of the CoDA-NO Layer

Full output concatenation and $\mathcal{I}_{per}$

$$o = [o^1, o^2, \ldots, o^T] : \mathcal{D} \to \mathbb{R}^{T \cdot d_v}$$

After multi-head attention, the full output o is formed by codomain-wise concatenation. The CoDA-NO layer then applies $\mathcal{I}_{per}$ (Eq. (3)) to o, followed by a residual connection and normalisation, to produce the next latent function. When operating on a uniform grid, the operators $\mathcal{K}^h$, $\mathcal{Q}^h$, $\mathcal{V}^h$, $\mathcal{M}$, and $\mathcal{I}$ are all implemented as FNOs.

Function Space Normalisation

Equation (7) — Mean and Standard Deviation of a Token Function

Equation (7): Functional Statistics

$$\mu^j = \int_{\mathcal{D}} w^j(x)\,\mathrm{d}x, \qquad \sigma^j = \left(\int_{\mathcal{D}} \left(w^j(x) - \mu^j\right)^{\circ 2} \mathrm{d}x \right)^{\circ \frac{1}{2}}. \tag{7}$$

Symbols in Equation (7)

$\mu^j \in \mathbb{R}^{d'}$	The **functional mean** of token w^j — the integral of w^j over the domain. Generalises the standard sample mean to functions.
$\sigma^j \in \mathbb{R}^{d'}$	The **functional standard deviation** of token w^j. Each component is the L^2 norm of the centred function $(w^j - \mu^j)$, computed channel-by-channel.
$(\cdot)^{\circ 2}$	**Elementwise (Hadamard) square**: for $v \in \mathbb{R}^{d'}$, $(v)^{\circ 2} = (v_1^2, \ldots, v_{d'}^2)$. Applied pointwise at each $x \in \mathcal{D}$, then integrated.
$(\cdot)^{\circ \frac{1}{2}}$	**Elementwise square root**: the component-wise square root of a vector. Applied after integration to recover the standard deviation per channel.
$w^j(x) - \mu^j$	Centring: the mean μ^j (a vector in $\mathbb{R}^{d'}$) is subtracted from $w^j(x)$ pointwise for each x, analogous to centring a random variable.

The Normalisation Operator

Norm Operator

$$\mathrm{Norm}[w^j](x) = \left(\mathbf{g} \oslash \sigma^j\right) \odot \left(w^j(x) - \mu^j\right) + \mathbf{b}.$$

Symbols in the Norm Operator

$\mathbf{b} \in \mathbb{R}^{d'}$	**Learnable bias** (shift) vector. One parameter per channel.
$\mathbf{g} \in \mathbb{R}^{d'}$	**Learnable gain** (scale) vector. One parameter per channel.
$\oslash$	**Elementwise (Hadamard) division**: $(\mathbf{g} \oslash \sigma^j)_k = g_k / \sigma^j_k$.
$\odot$	**Elementwise (Hadamard) multiplication**: $(u \odot v)_k = u_k v_k$.
$\mathbf{g} \oslash \sigma^j$	Scales the normalised function by a learnable gain, divided by the per-channel standard deviation. This is the *affine re-parameterisation* step.
$w^j(x) - \mu^j$	**Centred** token value at x.
$+\mathbf{b}$	Adds the learnable bias after scaling.

Connection to instance normalisation. The formula mirrors the classical *instance normalisation* formula $\hat{x} = \frac{x-\mu}{\sigma}$, but here μ and σ are computed as *integrals* over the continuous domain $\mathcal{D}$ rather than empirical statistics over a discrete batch or sequence. This ensures the normalisation is *discretisation-invariant*: it gives the same result regardless of how $\mathcal{D}$ is discretised.

Variable Specific Positional Encoding (VSPE)

VSPE Symbols

$e^i : \mathcal{D} \to \mathbb{R}^{d_{\text{en}}}$	The **positional encoder** for physical variable i. It produces a d_{en}-dimensional encoding at every point $x \in \mathcal{D}$, capturing both spatial position and the *identity* of variable i.
d_{en}	The **encoding dimension**; number of channels in the positional encoding.
$i \in \{1, \dots, d_{\text{in}}\}$	Variable index.
$\tilde{a}^i = [a^i, e^i]$	**Extended input function** for variable i: the scalar field a^i concatenated with the d_{en}-dim encoding e^i, giving $\tilde{a}^i : \mathcal{D} \to \mathbb{R}^{d_{\text{en}}+1}$.
$\mathcal{P} : \{\tilde{a}^i : \mathcal{D} \to \mathbb{R}^{d_{\text{en}}+1}\} \to \{w_e^i : \mathcal{D} \to \mathbb{R}^D\}$	The **shared pointwise lifting operator**, mapping each extended input to a latent token with D channels. Shared across all variables, enforcing permutation equivariance.
$D > d_{\text{en}} + 1$	The lifted channel width; must exceed the input dimension to allow information expansion.
$\kappa^i \in \mathbb{C}^{d_{\text{en}} \times \frac{n}{2}}$	**Learnable Fourier coefficients** for variable i. These are complex-valued matrices of size $d_{\text{en}} \times n/2$, where n is the number of grid points.
$\text{iFFT}(\kappa^i)$	The **inverse Fourier transform** of κ^i, mapping the learnable frequency-domain coefficients back to a real-space signal of length n. This produces the positional encoding as a learnable spectral signal.
$\tilde{a}^i \leftarrow \text{concatenate}(a^i, \text{iFFT}(\kappa^i))$	Algorithm step: construct the extended input by appending the iFFT-generated encoding to the raw variable field.

Equation (8) — Lifted Latent Function

Equation (8): Lifted Latent Function

$$w = [w_e^1, \dots, w_e^{d_{\text{in}}}] : \mathcal{D} \to \mathbb{R}^{D \cdot d_{\text{in}}}. \tag{8}$$

Symbols in Equation (8)

$w_e^i = \mathcal{P}[\tilde{a}^i]$	Lifted latent representation of variable i, obtained by passing the extended input through the pointwise MLP $\mathcal{P}$.
w	The **full lifted latent function**, formed by codomain-wise concatenation of all d_{in} lifted tokens. This is the input to the stack of CoDA-NO layers.
$D \cdot d_{\text{in}}$	Total channel count of the latent function. Each of the d_{in} variables contributes D channels.
$d = D \cdot d_{\text{in}}$	Shorthand used in the text for the total latent dimension. For permutation equivariance, d' (per-token width) must divide d, i.e., $T \mid D$.

Algorithm 1 — Pseudocode Walkthrough

Algorithm 1 in the paper illustrates CoDA-NO applied to three variables a^1, a^2, a^3 on a uniform 1-D grid. Below is a line-by-line mathematical explanation.

Inputs and VSPE

Algorithm Inputs

$$a^1, a^2, a^3 \in \mathbb{R}^{1\times n}$$

Three physical variable fields, each evaluated at n grid points.

Learnable Fourier Coefficients:

$$\kappa^1, \kappa^2, \kappa^3 \in \mathbb{C}^{d_{\text{en}}\times \frac{n}{2}}$$

One complex coefficient matrix per variable. The factor $n/2$ comes from the one-sided Fourier spectrum of a real-valued signal of length n.

Extended inputs:

$$\tilde{a}^i \leftarrow \text{concatenate}\big(a^i,\ \text{iFFT}(\kappa^i)\big), \quad i = 1, 2, 3.$$

Lifting (pointwiseMLP):

$$w^i \leftarrow \text{pointwiseMLP}(\tilde{a}^i) : \mathbb{R}^{(d_{\text{en}}+1)\times n} \to \mathbb{R}^{D\times n}, \quad i = 1, 2, 3.$$

Each $w^i \in \mathbb{R}^{D\times n}$ is a discretised token function on the grid.

Codomain Attention Block (repeated $\times L$)

Key/Query/Value via FNO

$$k^i \leftarrow \text{FNO}_k(w^i), \quad q^i \leftarrow \text{FNO}_q(w^i), \quad v^i \leftarrow \text{FNO}_v(w^i), \quad i \in \{1, 2, 3\}.$$

Three separate FNO modules (with shared weights across variable index i) produce key, query, and value functions for each variable.

Attention Coefficient Matrix M

$$\mathbf{M}[i, j] \leftarrow \langle q^i, k^j \rangle = \int_{\mathcal{D}} \langle q^i(x), k^j(x) \rangle \, \mathrm{d}x, \quad i, j \in \{1, 2, 3\}.$$

$\mathbf{M} \in \mathbb{R}^{3\times 3}$: the raw attention logit matrix. Entry (i, j) measures the affinity of query i toward key j.

$$\mathbf{M} \leftarrow \text{Softmax}\left(\frac{\mathbf{M}[i, j]}{N}\right),$$

where N is a normalisation constant (analogous to τ in Eq. (5)), typically $N = \sqrt{d_k}$ or the number of grid points. Softmax is applied *row-wise*: each row i becomes a

probability vector over j.

Attention Output

$$o^i \leftarrow \sum_{j\in\{1,2,3\}} v^j \times \mathbf{M}[i,j], \quad i \in \{1,2,3\}.$$

The output for variable i is a **weighted sum of value functions**. The scalar weight $\mathbf{M}[i,j]$ scales the entire function v^j (pointwise multiplication at each grid point x).

Normalise + Residual

$$o^i \leftarrow \text{norm}(o^i) + w^i, \quad i \in \{1,2,3\}.$$

norm applies Eq. (7) function-space normalisation. The **residual connection** $+w^i$ stabilises training (analogous to the skip connection in a standard transformer).

Integral update via FNO:

$$w^1, w^2, w^3 \leftarrow \text{FNO}_{\mathcal{I}}(o^1),\ \text{FNO}_{\mathcal{I}}(o^2),\ \text{FNO}_{\mathcal{I}}(o^3).$$

Applies the permutation-equivariant integral operator $\mathcal{I}_{per}$ (Eq. (3)) via shared-weight FNOs. This block repeats L times.

Projection

Projection back to physical space

$$u^i \leftarrow \text{pointwiseMLP}(w^i) : \mathbb{R}^{D\times n} \to \mathbb{R}^{1\times n}, \quad i = 1,2,3.$$

The projection is the inverse of the lifting step. Each latent token w^i is projected back to a scalar solution field u^i at the n grid points.

Architecture: Encoding, Decoding, and Training

Encoding on Non-Uniform Geometries (GINO)

GINO Notation

$\{a(x_i^{\text{in}})\}_{i=1}^n$	Evaluations of the input function at n **mesh points** $\{x_i^{\text{in}}\}_{i=1}^n \subset \mathcal{D}_{\text{in}}$ (possibly non-uniform).
$\mathcal{D}_{\text{in}}$	The (irregular) input mesh domain.
GNO_{per}	Permutation-equivariant **Graph Neural Operator**: maps the irregular mesh evaluations to a uniform latent grid $\{x_l^{\text{grid}}\}_{l=1}^{n'}$.
w_0	Initial latent function on the uniform grid, produced by GNO_{per} after concatenating VSPEs.
w_l	Latent function after l stacked CoDA-NO layers applied to w_0. Serves as the **encoded representation** of the input.
l	The number of stacked CoDA-NO layers in encoder (and decoder).
$\{u(x_i^{\text{out}})\}_{i=1}^{n'}$	Predicted solution values on the (arbitrary) output grid, produced by the decoder's GNO_{per} from w_L.

Two-Stage Training

Pre-training and Fine-tuning

Encoder	The first block of l CoDA-NO layers. Transforms w_0 into a rich latent representation.
Reconstructor	Decoding component used **only during pre-training**. Mirrors the encoder and reconstructs masked inputs.
Predictor	Randomly initialised decoding component used during **fine-tuning** in place of the Reconstructor. Predicts the solution u from the latent code.
Masking	During pre-training, a fraction of mesh points are set to zero (spatial masking), and entire variable channels may also be zeroed (channel/codomain masking).
VSPEs (fine-tuning)	For variables present in the fine-tuning PDE but absent in pre-training, only the corresponding VSPE parameters κ^i and encoder are newly trained; all shared parameters are copied from pre-training.

Complete Symbol Reference Table

Symbol	Meaning	Space / Type
$\mathcal{D}$	Physical input domain	$\subset \mathbb{R}^n$
a	Input function (PDE condition)	$\mathcal{D} \to \mathbb{R}^{d_{\text{in}}}$
u	Output/solution function	$\mathcal{D} \to \mathbb{R}^{d_{\text{out}}}$
d_{in}	Number of input physical variables	$\in \mathbb{N}$
d_{out}	Number of output physical variables	$\in \mathbb{N}$
a^i	i-th scalar component of a	$\mathcal{D} \to \mathbb{R}$
θ	Parameters of pointwise operator	$\in \mathbb{R}^p$
ϕ	Parameters of kernel/integral operator	$\in \mathbb{R}^q$
h_θ	Pointwise neural network	$\mathbb{R}^{d_f} \to \mathbb{R}^{d_g}$
k_ϕ	Integral kernel function	$\mathcal{D} \times \mathcal{D} \to \mathbb{R}^{d_g \times d_f}$
$\mathcal{H}$	Pointwise operator	Function space map
$\mathcal{T}$	Integral operator	Function space map
$\mathcal{I}$	Shared integral operator in $\mathcal{I}_{per}$	Function space map
$\mathcal{I}_{per}$	Permutation-equivariant integral operator	Function space map
$\mathcal{H}_{per}$	Permutation-equivariant pointwise operator	Function space map
w^j	j-th token function	$\mathcal{D} \to \mathbb{R}^{d'}$
w_e^i	Lifted token for variable i	$\mathcal{D} \to \mathbb{R}^D$
T	Number of tokens (physical variables)	$\in \mathbb{N}$
d'	Per-token channel width (d/T)	$\in \mathbb{N}$
D	Lifted channel width per variable	$\in \mathbb{N}$
$\mathcal{K}$	Key operator	$\mathcal{W} \to \{\mathcal{D} \to \mathbb{R}^{d_k}\}$
$\mathcal{Q}$	Query operator	$\mathcal{W} \to \{\mathcal{D} \to \mathbb{R}^{d_q}\}$
$\mathcal{V}$	Value operator	$\mathcal{W} \to \{\mathcal{D} \to \mathbb{R}^{d_v}\}$
k^j	Key function for token j	$\mathcal{D} \to \mathbb{R}^{d_k}$
q^j	Query function for token j	$\mathcal{D} \to \mathbb{R}^{d_q}$
v^j	Value function for token j	$\mathcal{D} \to \mathbb{R}^{d_v}$
$d_k = d_q$	Key/query dimension	$\in \mathbb{N}$
d_v	Value dimension	$\in \mathbb{N}$
$\langle \cdot, \cdot \rangle$	$L^2(\mathcal{D}, \mathbb{R}^{d_k})$ inner product	Scalar
τ	Temperature hyperparameter	$\in \mathbb{R}_{>0}$
o^j	Attention output token for variable j	$\mathcal{D} \to \mathbb{R}^{d_v}$
H	Number of attention heads	$\in \mathbb{N}$
c^j	Multi-head concatenated output for token j	$\mathcal{D} \to \mathbb{R}^{Hd_v}$
$\mathcal{M}$	Multi-head mixing operator	Function space map
μ^j	Functional mean of w^j	$\in \mathbb{R}^{d'}$